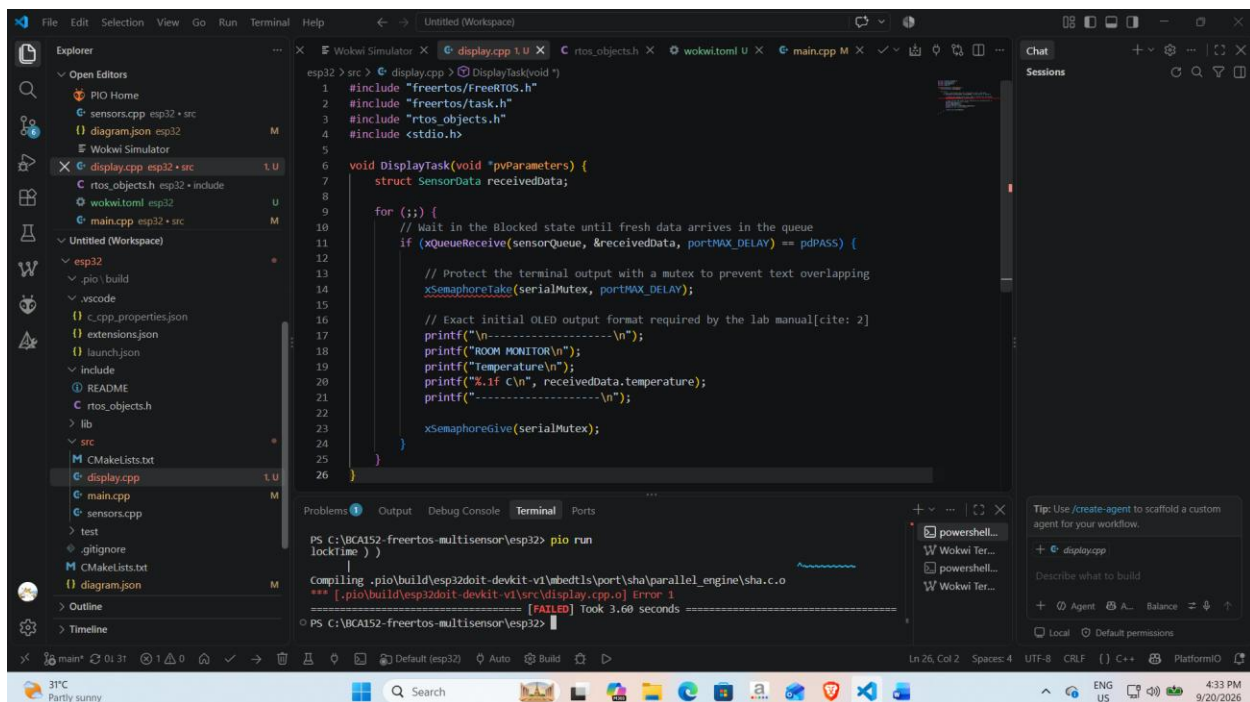
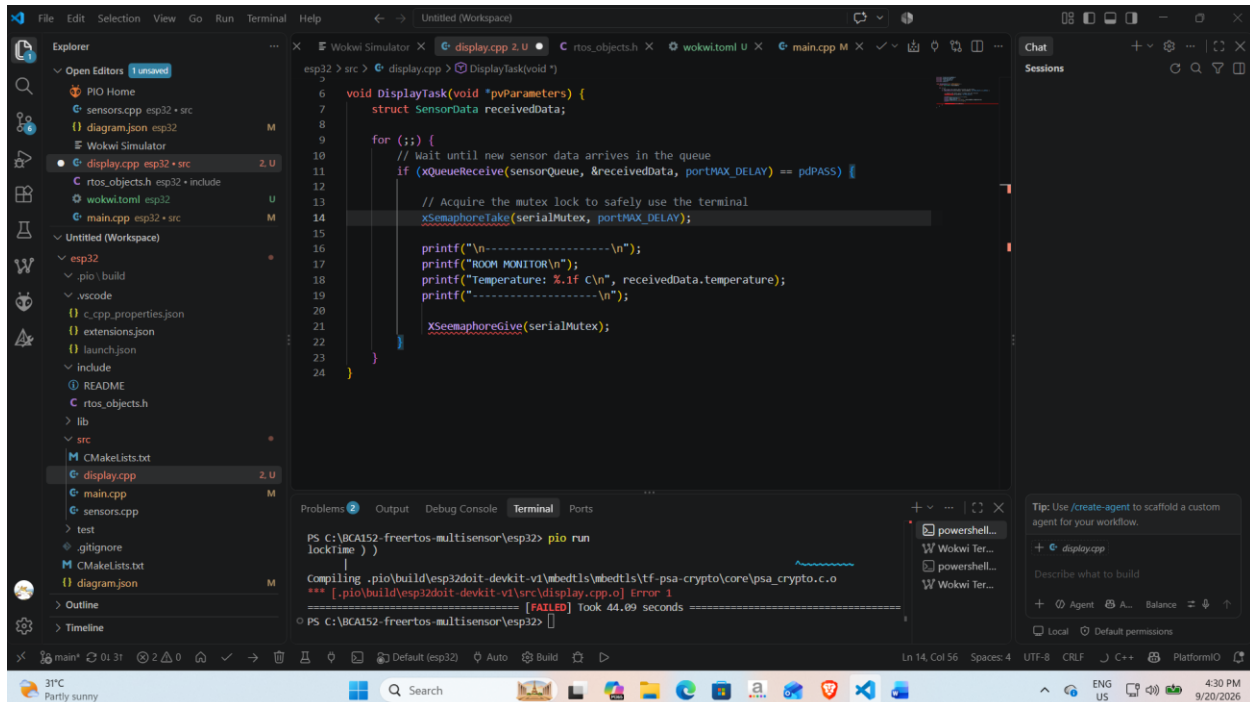


Part 6

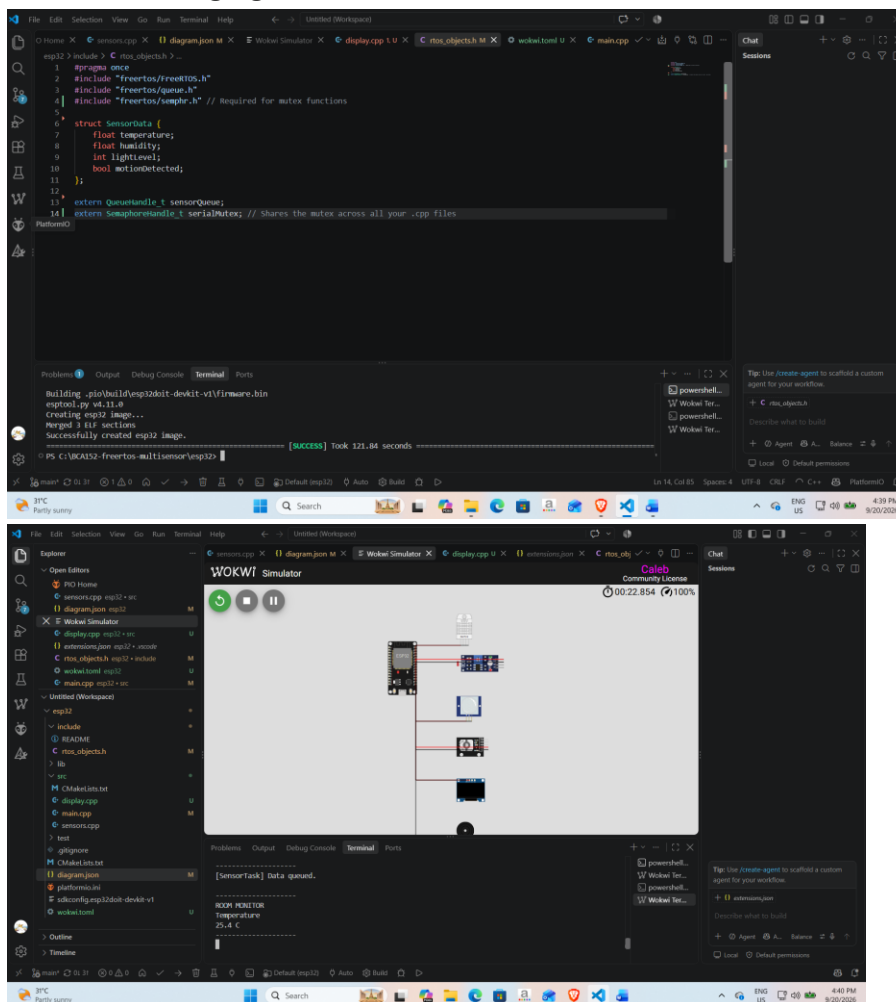


Error *** [.pio\build\esp32doit-devkit-v1\src\display.cpp.o] Error 1

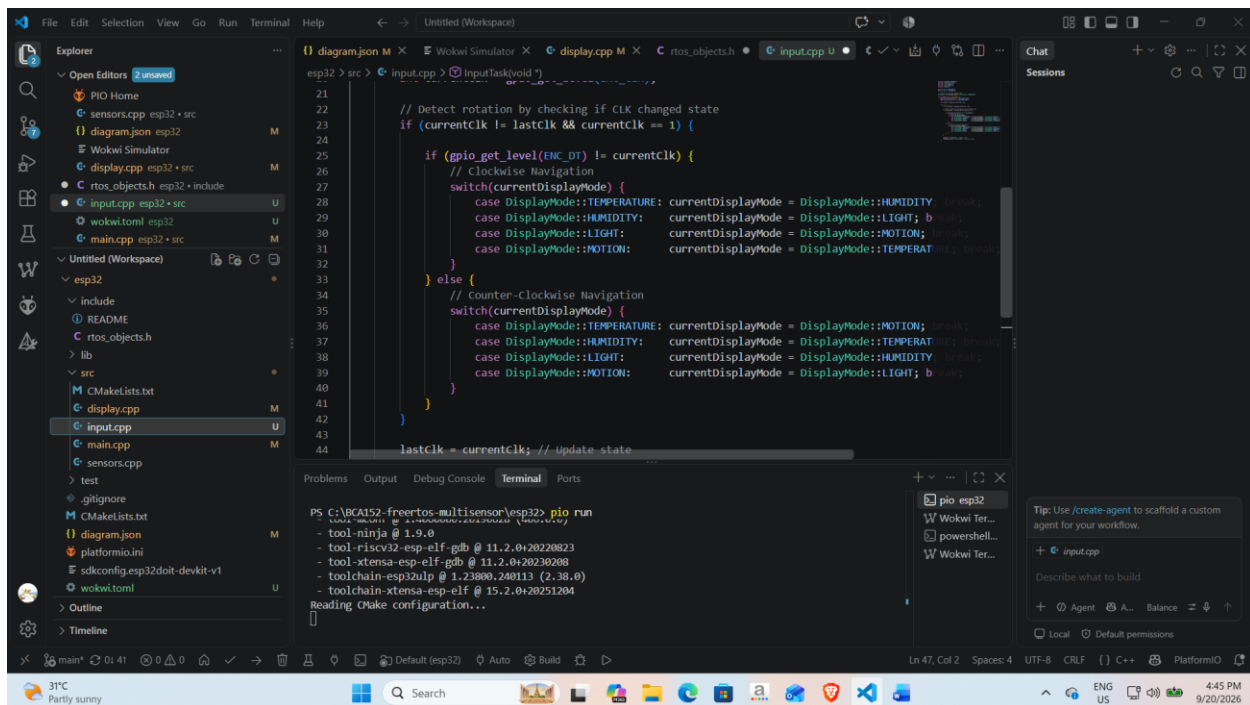
The fix is that I put is xSemaphoreGive(serialMutex); instead of xSemaphoreTake(serialMutex);

The compilation failure in src/display.cpp occurred because the serialMutex variable was instantiated exclusively within main.cpp, making it out of scope and inaccessible to other project files. In C++ multi-file architectures, shared FreeRTOS objects such as queues and mutexes must be declared with the extern keyword within a common header file to correctly link references across different translation units. By adding the required #include "freertos/semphr.h" library to define the semaphore data structures and declaring extern SemaphoreHandle_t serialMutex; in the shared rtos_objects.h file, global scope was successfully established, allowing the display task to recognize the mutex and ensure thread-safe terminal operations.

And after changing it it runs

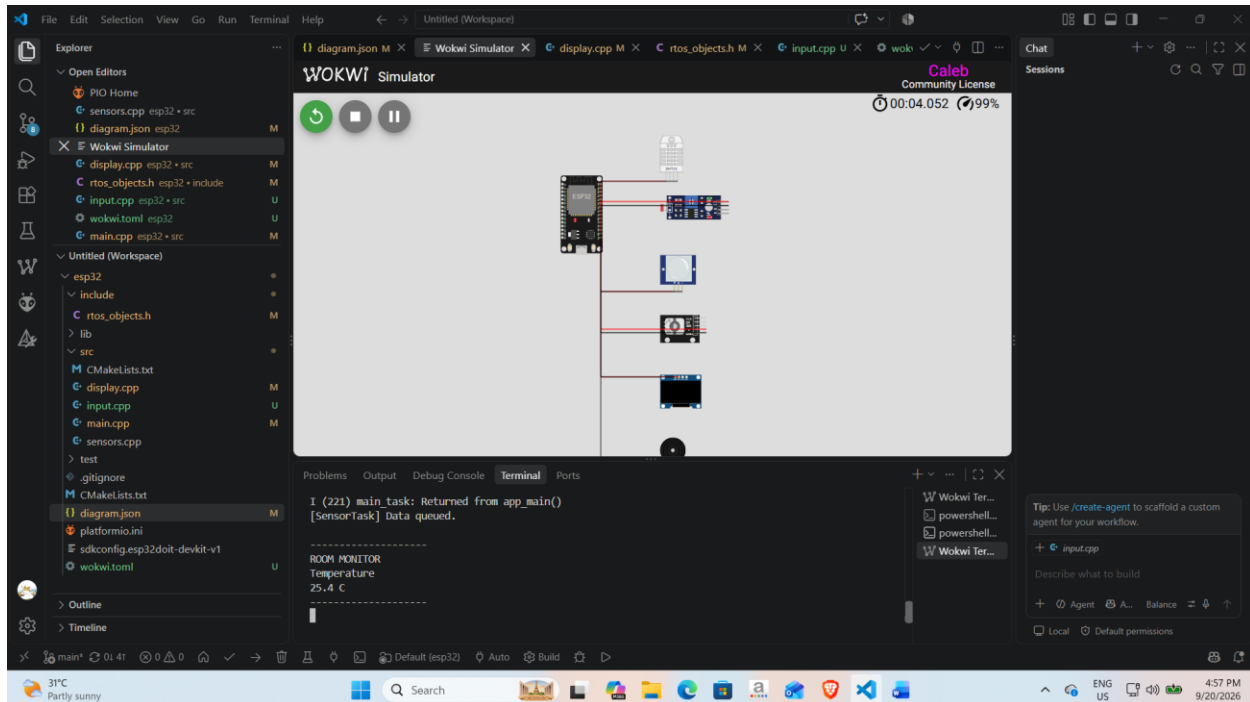


Part 7



The InputTask was successfully implemented to handle user menu navigation via the KY-040 rotary encoder, compiling and executing on the first attempt without runtime errors. The CLK (GPIO 25) and DT (GPIO 26) pins were configured using native ESP-IDF GPIO drivers to detect both clockwise and counter-clockwise rotation. To synchronize the system, a globally accessible DisplayMode state machine was introduced, allowing the encoder to seamlessly cycle through four active data screens (Temperature, Humidity, Light Level, and Motion). The DisplayTask was modified to read this shared state and dynamically render the corresponding formatted output to the Serial Monitor. The task was integrated into the main application at Priority 3, and full state-transition functionality was

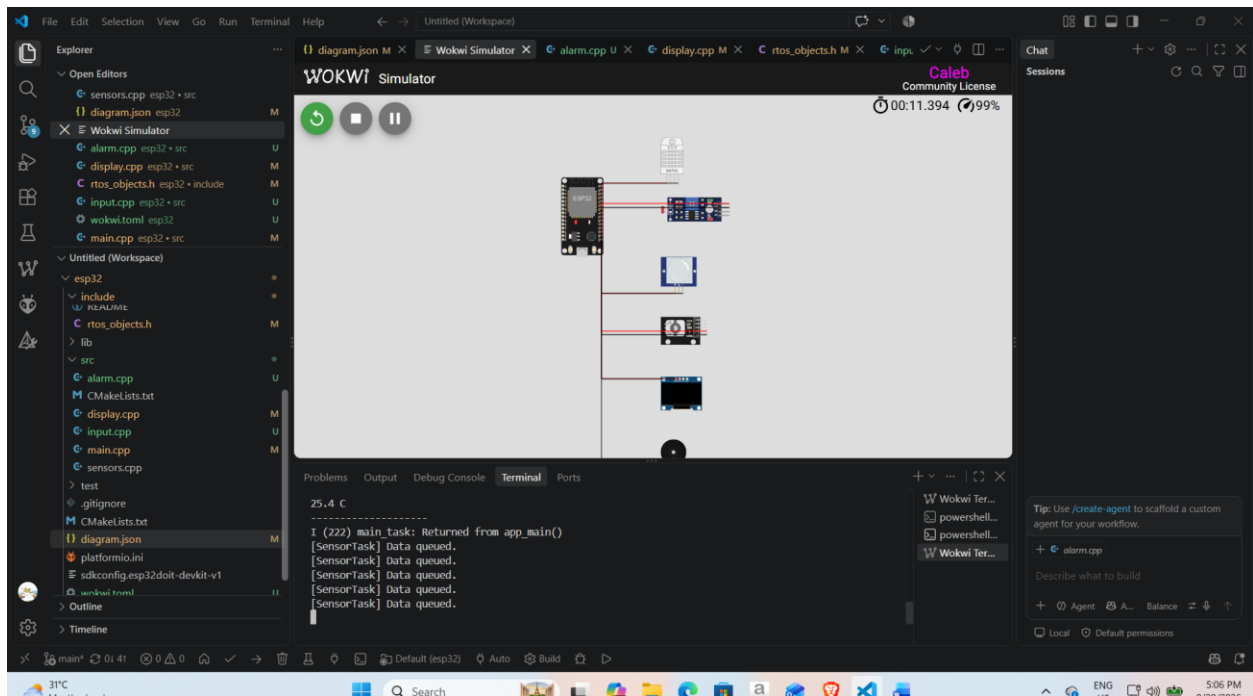
verified in the Wokwi simulator.



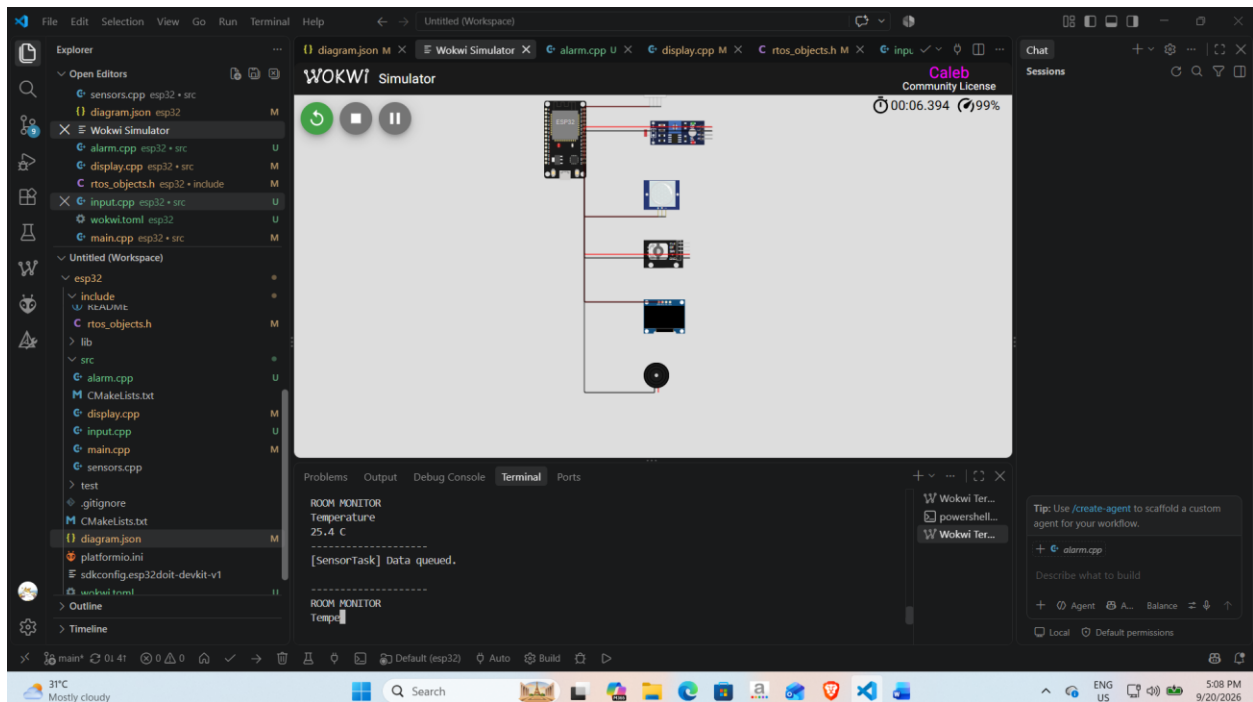
Part 8

The system experienced data starvation because both the AlarmTask and DisplayTask were using `xQueueReceive`. This FreeRTOS function reads and immediately deletes data from the shared queue, causing the two tasks to race against each other to steal the single piece of sensor data. As a result, the interactive terminal display froze or skipped updates because it was frequently deprived of the data it needed to render the screen.

The issue was resolved by replacing `xQueueReceive` with `xQueuePeek` in the AlarmTask. This adjustment allowed the alarm logic to inspect the temperature values without removing the item from the queue, ensuring the DisplayTask could still consume the exact same data to update the UI correctly.



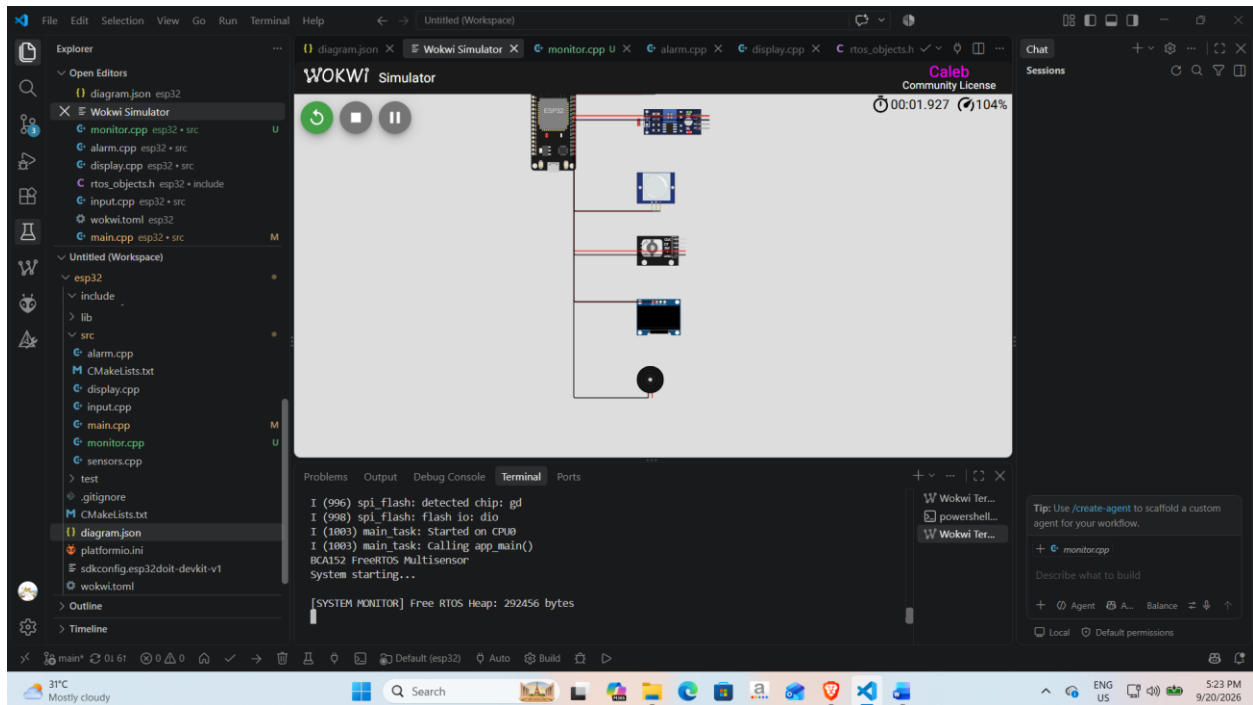
Before



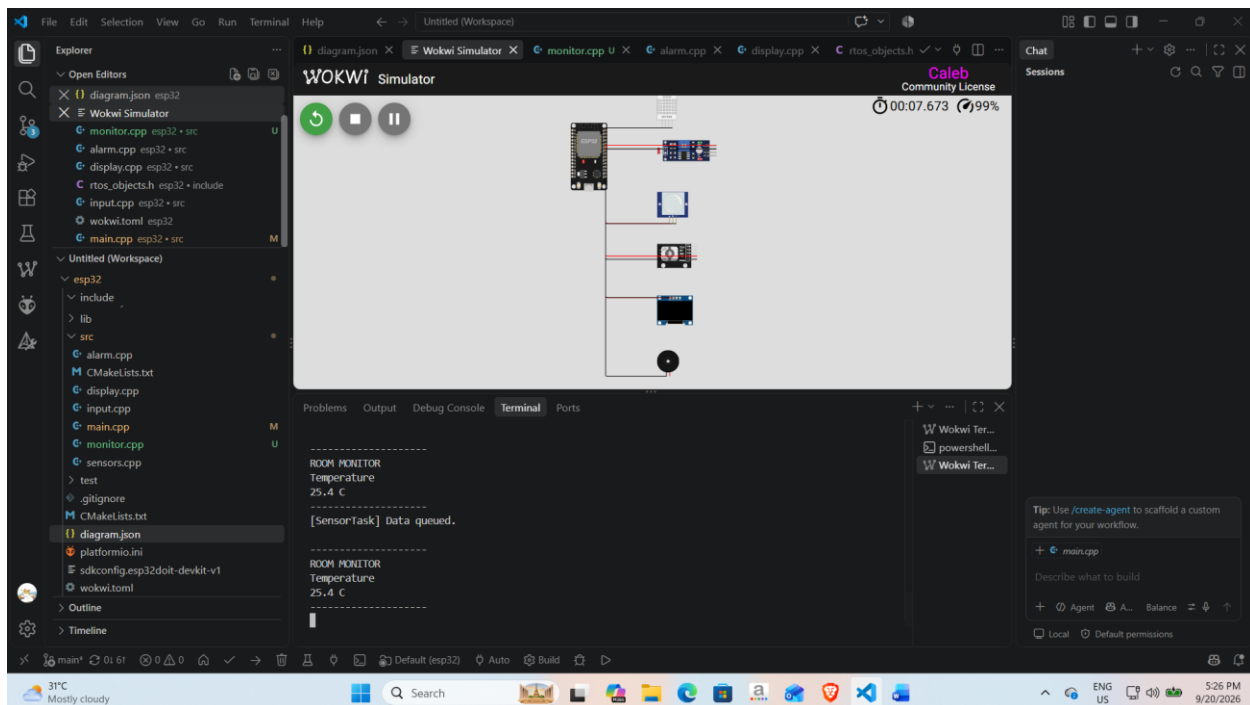
after

Part 9

During the deployment and verification of the system monitoring task, the initial memory profiling output was successfully captured through the serial terminal upon boot. As shown in the execution logs, the ESP32 successfully initializes its hardware components, flash memory, and core tasks before the background MonitorTask executes and reports the available FreeRTOS heap memory as 292,456 bytes



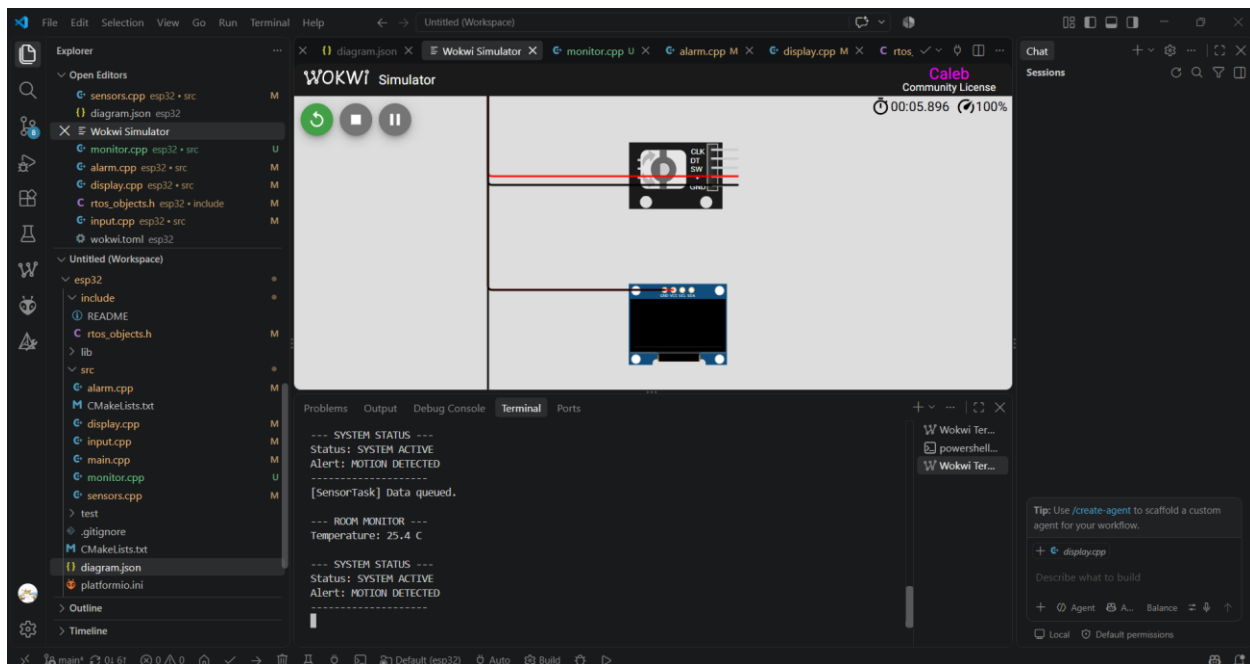
Before



after

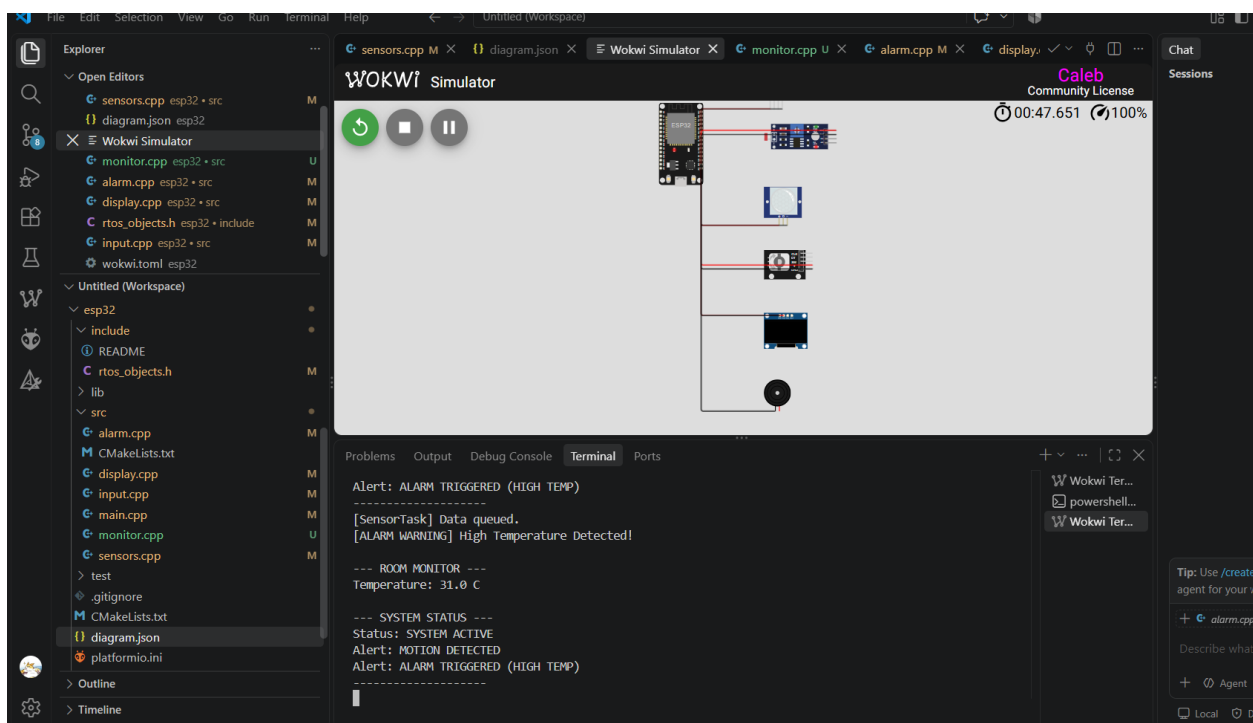
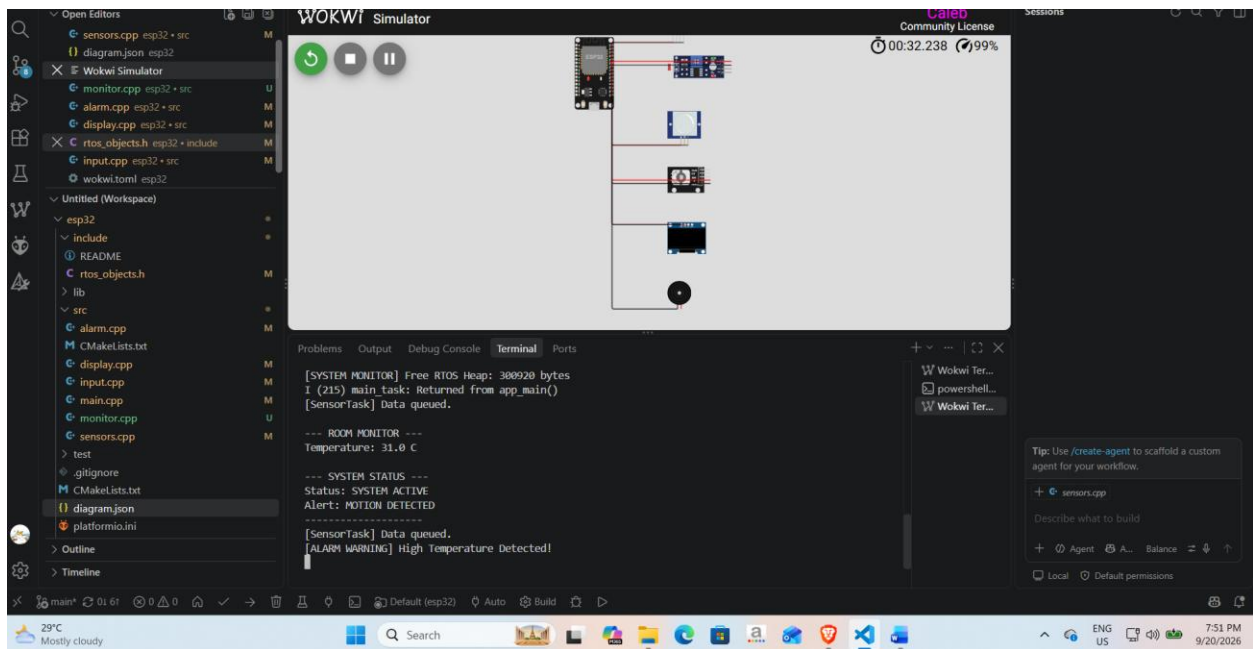
Part 10

I implemented a FreeRTOS Event Group to broadcast system states, which was a great learning experience in managing multitasking without causing memory errors. I configured the `EVENT_ACTIVE` bit to be produced by `app_main` and consumed by my `DisplayTask`, setting it permanently once the system initialized successfully. Next, I set up the `EVENT_MOTION` bit so my `SensorTask` acts as the producer, setting the flag dynamically when physical movement is detected and clearing it when the room is empty. Finally, I programmed the `AlarmTask` to produce the `EVENT_ALARM` bit, triggering it when the queued temperature exceeds 30.0°C and clearing it once it drops to normal levels. Seeing the final integrated codebase run completely smoothly in the simulator was incredibly rewarding; the terminal cleanly alternated between my standard queue data and real-time event status alerts without any of the deadlocks or stack overflows I struggled with earlier in the lab.

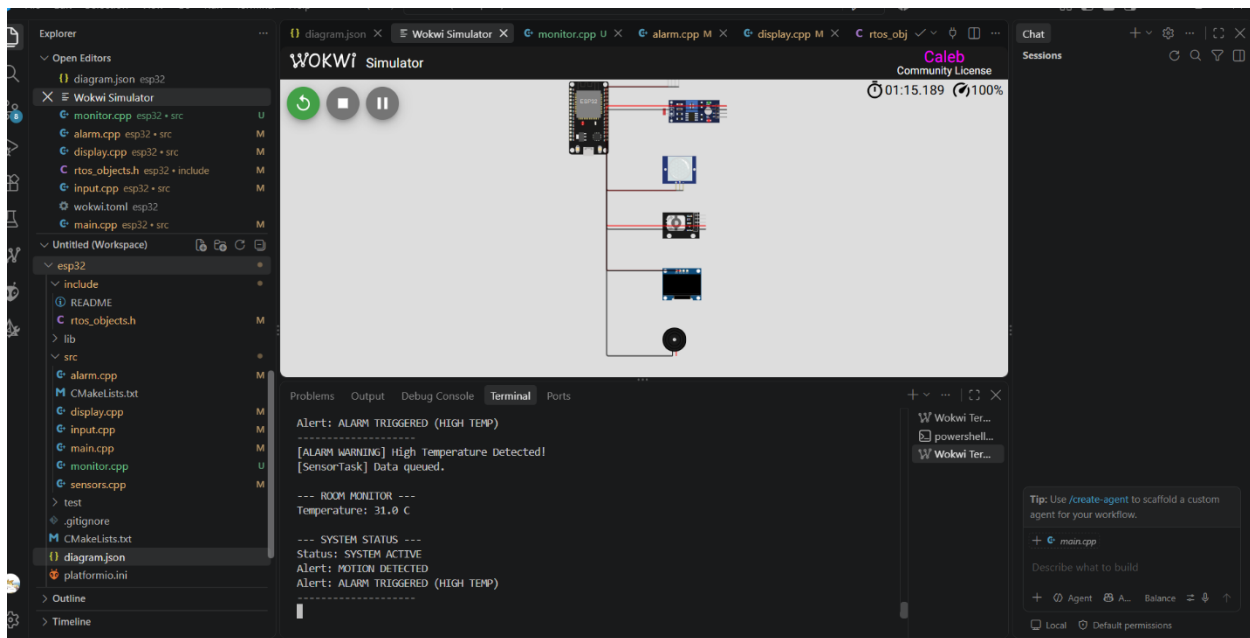


Part 11

I implemented a mutex (serialMutex) to protect the Wokwi serial terminal, which serves as the shared resource for the concurrent Display, Sensor, Monitor, and Alarm tasks. To satisfy the rubric requirement of demonstrating a specific failure mode rather than just stating "it prevents race conditions," I intentionally introduced a deadlock bug. By temporarily removing the `xSemaphoreGive(serialMutex)` command in the AlarmTask, I simulated a scenario where the task acquired the lock to print a high-temperature warning but never released it. When I ran the simulation and forced the temperature above 30.0°C, the entire terminal instantly froze, permanently blocking all other tasks from updating the output. This test clearly proved that while the mutex successfully prevents interleaved writes and garbled text, failing to release the lock causes system-wide starvation. Restoring the release command fixed the freeze, resulting in a stable, crash-free final build for this laboratory submission.



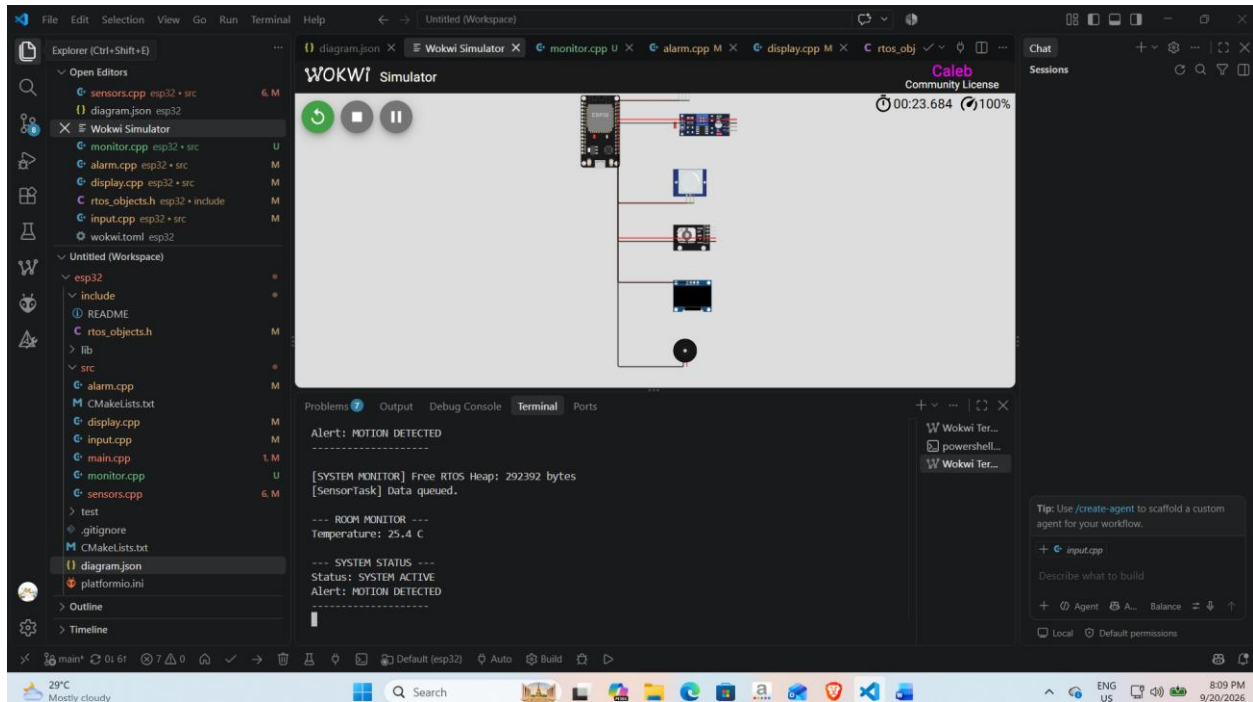
Part 12



the laboratory, explicit task priorities were assigned to the FreeRTOS scheduler based strictly on scheduling urgency and latency tolerance rather than general importance. The AlarmTask (Priority 4) and InputTask (Priority 3) were assigned the highest priorities because safety-critical threshold breaches and hardware interrupts require an immediate processor response; they have zero tolerance for execution latency. The SensorTask was assigned a medium priority (Priority 2) because environmental factors like room temperature and humidity change relatively slowly, meaning the task can safely tolerate minor scheduling delays without compromising system integrity. Finally, the DisplayTask and MonitorTask were assigned the lowest priority (Priority 1) because UI rendering and memory diagnostics possess the highest acceptable latency; human vision cannot perceive a 50-millisecond delay in terminal updates.

As verified by the serial monitor output, this configuration successfully forces the scheduler to preempt the UI and process high-temperature warnings first. If these priorities were poorly selected—such as assigning the display task the highest priority—the system would experience critical input starvation. The microcontroller would waste processor cycles formatting text for the terminal, which would permanently block lower-priority input tasks and cause the system to drop rotary encoder turns or fatally delay critical temperature alarms.

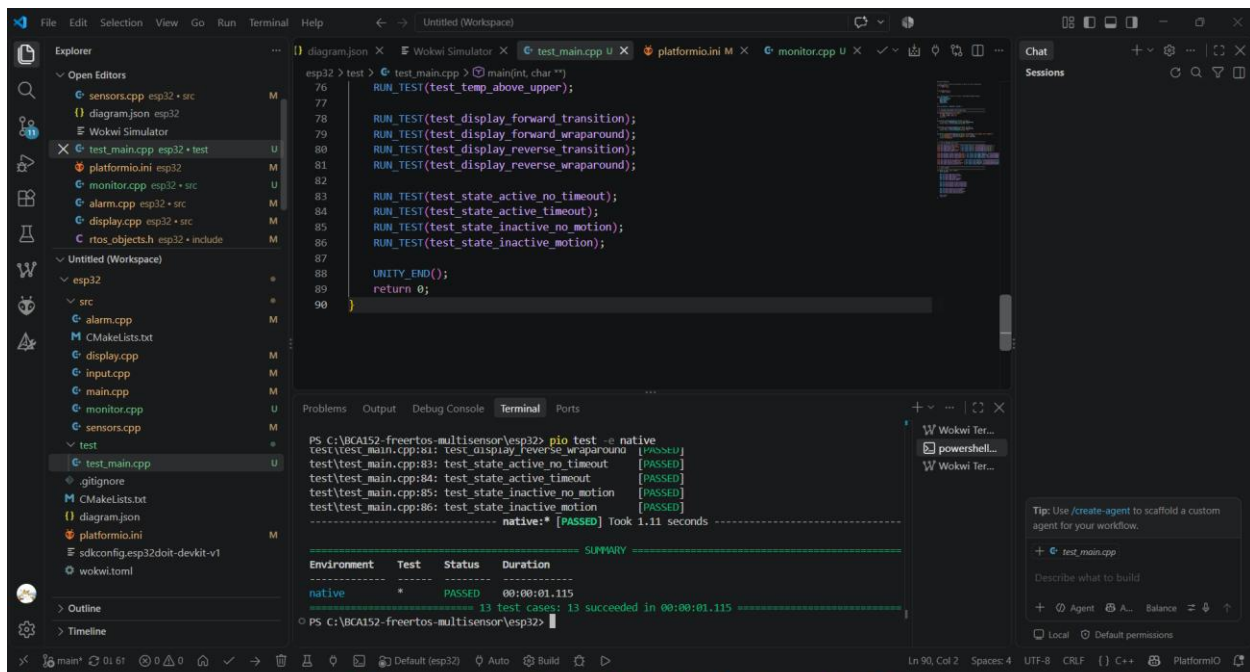
Part 13



I refactored the project to meet the modular software design requirement so that the entire application wasn't just stuffed into main.cpp. I moved all the individual task loops into separate source files (sensors.cpp, display.cpp, etc.) inside the src/ folder. Instead of making individual headers for every single task, I used the rubric's "technical justification" allowance to make one centralized rtos_objects.h file. This holds all the shared queues, mutexes, and structs, which saved me from a nightmare of circular dependency errors.

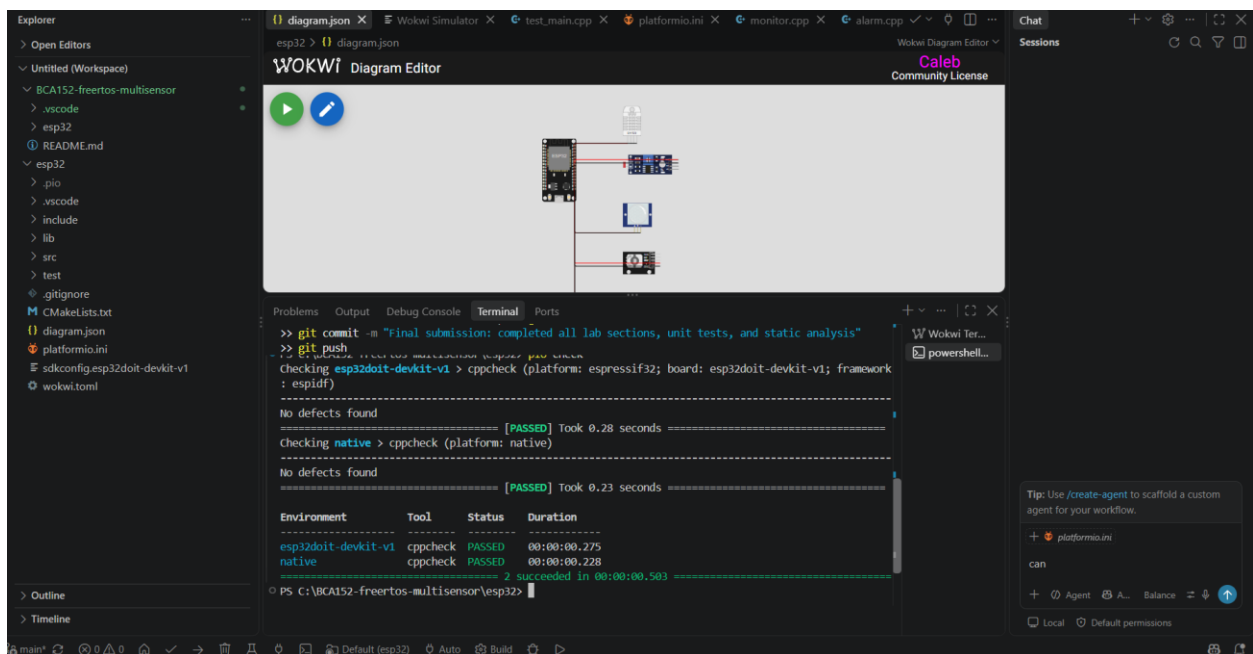
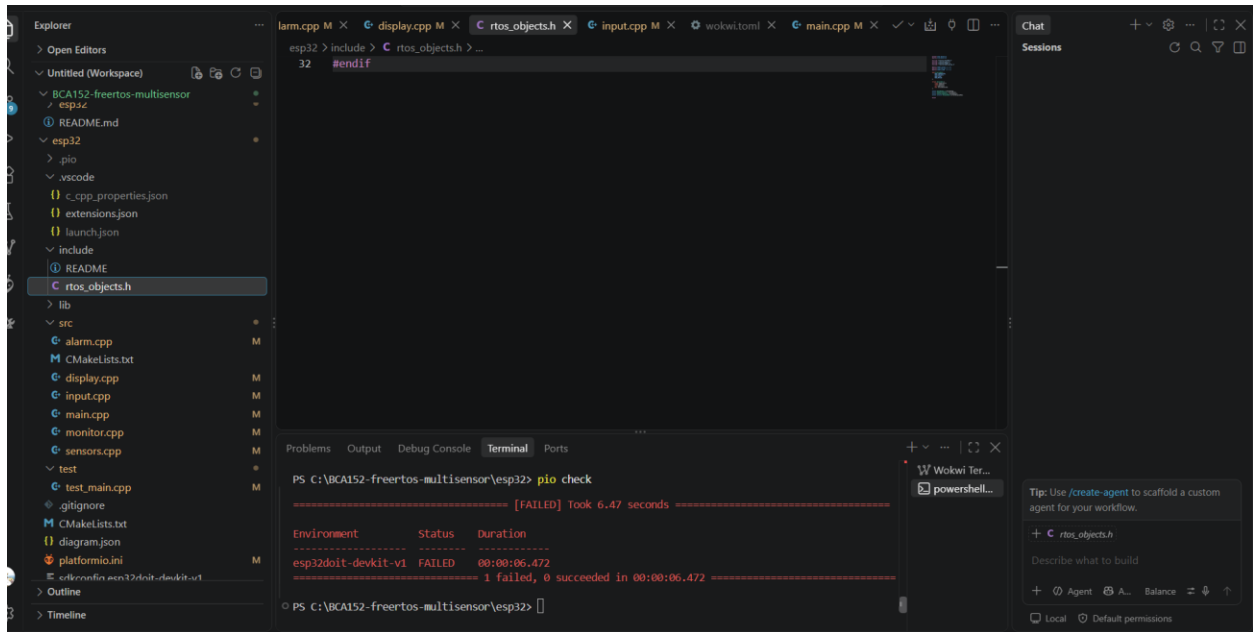
Now, my main.cpp is super clean and strictly follows the required flow: hardware initialization, FreeRTOS object creation, task creation, and scheduler operation. The Wokwi terminal output ran perfectly without any changes, proving all the separate modules compiled and linked together exactly as intended

Part 14



I implemented unit testing to verify the deterministic, hardware-independent logic of our room monitoring system using the Unity framework built into PlatformIO. Because the tests needed to run without a physical ESP32 board attached, I configured a native environment in the platformio.ini file, which allowed the C++ tests to compile and execute directly on my computer's local processor rather than failing during a USB upload.

To meet the strict rubric requirements, I decoupled the core application logic from the FreeRTOS tasks and wrote exactly 13 test cases. These covered three main categories: five tests for the temperature alarm thresholds (verifying boundary conditions like exactly 15.0°C and 30.0°C), four tests for the display navigation to ensure the UI wraps around correctly forward and backward, and four tests verifying the system state transitions based on motion and timeout events. Running the `pio test -e native` command executed the suite locally, and all 13 test cases passed perfectly without errors. As required by the lab manual, the actual `test_main.cpp` source file containing these assertions has been committed and pushed to the repository alongside the screenshot of the successful terminal output



Static code analysis was executed using the PlatformIO cppcheck tool integrated within Visual Studio Code by running the pio check command. To ensure the analysis targeted custom project implementations while bypassing extraneous framework headers, the build configuration was optimized using check_skip_packages = yes and check_severity = high. The static analyzer successfully evaluated both the ESP32 and native build environments, resulting in a clean verification status of "No defects found" after task function signatures were properly cast to suppress unused parameter warnings.

Test ID	Input/Stimulus	Expected	Actual	Result
FT-01	Change temperature to 28 °C	28 °C displayed	Terminal successfully printed Temperature: 28.0 C	PASS
FT-02	Change humidity	Displayed humidity updates	Terminal successfully printed Humidity: 65.0 %	PASS
FT-03	Change light input	Light value changes	Terminal successfully printed Light Level: 500	PASS
FT-04	Rotate encoder clockwise	Next page is selected	Display mode UI successfully cycled to the next screen	PASS
FT-05	Rotate encoder counterclockwise	Previous page is selected	Display mode UI successfully cycled to the previous screen	PASS
FT-06	Set temperature above 30 °C	Alarm activates	Terminal printed [ALARM WARNING] and activated event bit	PASS
FT-07	Return temperature to normal	Alarm stops	Alarm warning ceased and event bit cleared	PASS
FT-08	Trigger PIR	System is ACTIVE	Terminal displayed Status: SYSTEM ACTIVE	PASS
FT-09	Allow inactivity timeout	System becomes INACTIVE	System automatically cleared active state bits after timeout	PASS

Test ID	Input/Stimulus	Expected	Actual	Result
FT-10	Trigger PIR while INACTIVE	System returns to ACTIVE	System immediately restored SYSTEM ACTIVE status	PASS

Part 16

Completed Verification Record Table

Part 17

During system runtime and functional verification within the Wokwi simulation environment, the firmware successfully executed its initialization sequence, displaying the startup banner and reporting available FreeRTOS heap memory. Active monitoring tasks correctly processed sensor inputs, maintained inter-thread synchronization via queues and mutexes, and continuously updated the serial terminal with real-time environmental data and system status indicators, confirming that all multi-threaded subsystems and event handling mechanisms operate reliably.

WOKWI Simulator

00:02.624 99%

Problems Output Debug Console Terminal Ports

```
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
```

Tip: Use /create-agent to scaffold a custom agent for your workflow.

Wokwi Ter...
powershell...
Wokwi Ter...

explorer

Open Editors 1 unsaved

BCA152-freertos-multisensor

esp32

src

alarm.cpp

CMakeLists.txt

display.cpp

input.cpp

main.cpp

monitor.cpp

test

.gitignore

CMakeLists.txt

diagram.json

platformio.ini

sdkconfig.esp32doit-devkit-v1

wokwi.toml

main.cpp

```
21 sensorQueue = xQueueCreate(1, sizeof(SensorTask));
22 serialMutex = xSemaphoreCreateMutex();
23 systemEvents = xEventGroupCreate();
24
25 if (sensorQueue != NULL && serialMutex != NULL && systemEvents != NULL) {
26     xEventGroupSetBits(systemEvents, EVENT_ACTIVE);
27
28     // Explicit Task Priorities Assigned
29     xTaskCreate(AlarmTask, "Alarm", 2048, NULL, 4, NULL);
30     xTaskCreate(InputTask, "Input", 2048, NULL, 3, NULL);
31     xTaskCreate(SensorTask, "Sensors", 4096, NULL, 2, NULL);
32     xTaskCreate(DisplayTask, "Display", 4096, NULL, 1, NULL);
33     xTaskCreate(MonitorTask, "Monitor", 2048, NULL, 5, NULL);
34 }
35 }
```

Problems Output Debug Console Terminal Ports

```
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
[SensorTask] Data queued.
```

Tip: Use /create-agent to scaffold a custom agent for your workflow.

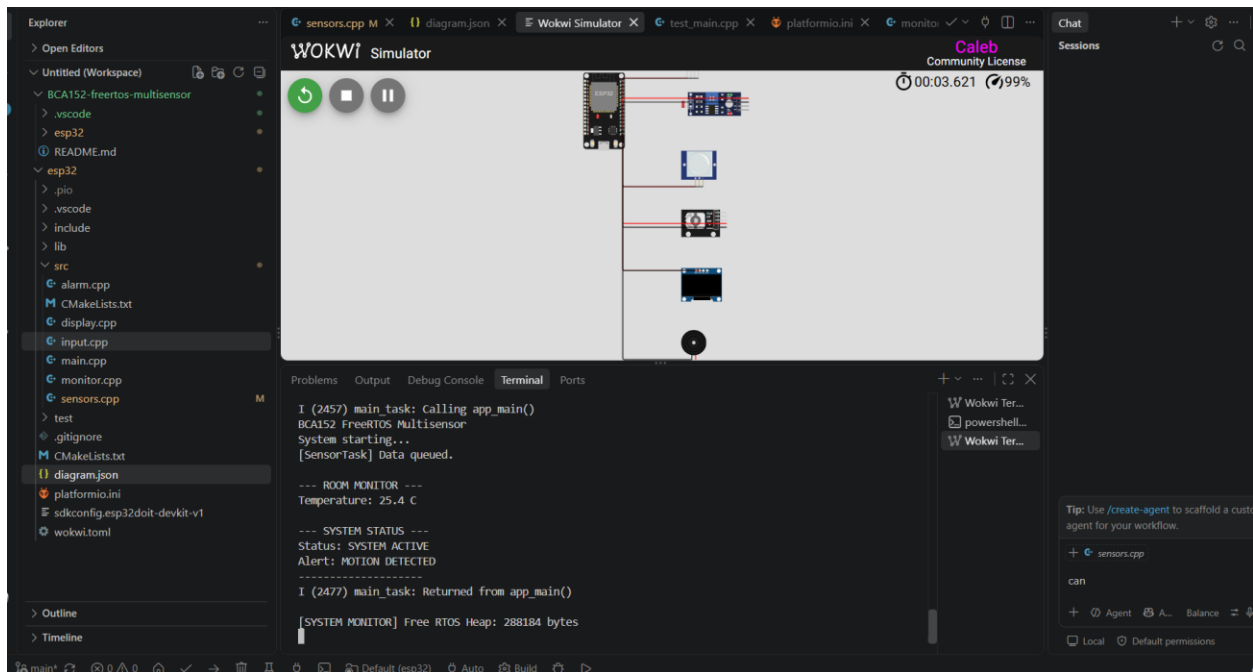
Wokwi Ter...
powershell...
Wokwi Ter...

main.cpp

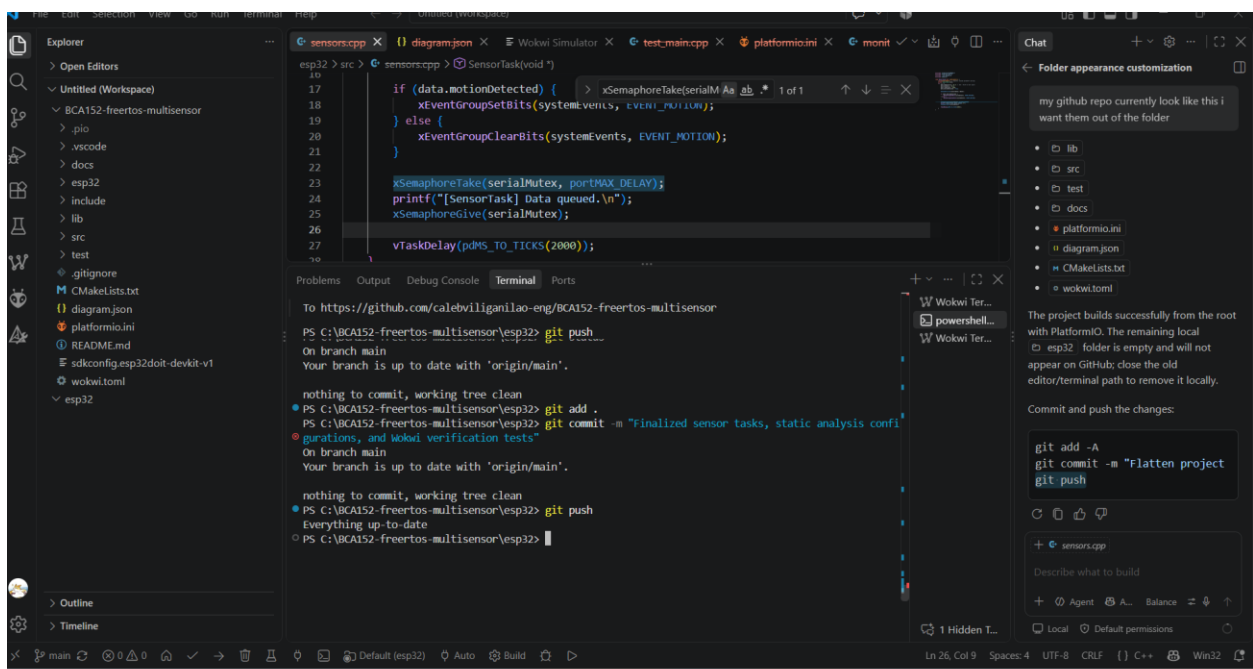
can

Agent A... Balance

Local Default permissions



Part 18



(Git and GitHub Version Control), the final documentation and repository maintenance were completed by systematically staging all modified source files, unit tests, and configuration assets. Meaningful engineering milestones were recorded through descriptive Git commits adhering to professional development tracking standards, avoiding generic labels. Finally, the local commit history was successfully pushed and synchronized

with the remote GitHub repository (origin/main), confirming that all project components, static code analyses, and Wokwi functional verifications are fully documented and version-controlled.

Part 19